Smart Invoice Extractor - Project Completion Report

Student Project Overview

This document confirms the successful completion of the project titled 'Smart Invoice Extractor – Invoice-to-JSON Converter using Multimodal Vision AI'. The objective of the project was to leverage OpenRouter's multimodal vision models to extract structured data from invoice images and convert them into JSON and tabular format.

Project Objective

To build an AI-powered system capable of analyzing invoice images and extracting structured data such as vendor details, invoice metadata, line items, and totals using multimodal vision models.

Key Features Implemented

- Upload invoice images (JPG/PNG).
- Convert image to Base64 format in the frontend.
- Send Base64 image to backend API for AI processing.
- Use OpenRouter multimodal vision model to analyze invoices.
- Extract structured JSON data including vendor, invoice number, dates, and line items.
- Display extracted data as a clean table in the frontend UI.
- Implement safe JSON parsing and validation.
- Add total verification logic to detect mismatches in invoice totals.
- Handle API rate limits with retry logic.
- Provide automated backend testing using Jest and Supertest.

Technology Stack

- Frontend: HTML, CSS, JavaScript
- Backend: Node.js, Express, TypeScript
- AI Model: OpenRouter Vision Model (qwen/qwen-2.5-vl-7b-instruct)
- Testing: Jest + Supertest
- Environment Configuration: dotenv

System Workflow

1. User uploads an invoice image via the frontend.

2. The image is converted into a Base64 string.
3. The Base64 image is sent to the backend API.
4. The backend sends the image and prompt to OpenRouter Vision Model.
5. The AI model analyzes the spatial layout of the invoice.
6. The AI extracts structured JSON data from the invoice.
7. The backend validates the JSON response.
8. Total verification logic checks for mismatches in invoice totals.
9. The frontend displays the extracted data in JSON and table format.

Testing

Automated tests were implemented using Jest and Supertest to validate the backend API functionality. Test cases verify correct API responses, error handling, and mock AI responses.

Error Handling

- Missing image input returns HTTP 400 error.
- Invalid JSON returned by AI is safely handled.
- API rate limit errors trigger automatic retries.
- Invalid or corrupted invoice images return fallback messages.

Githhub repo: BhoomiBliss/Smart-invoice-extractor: A TypeScript + OpenRouter app that converts invoice images into structured JSON data.

Conclusion

The Smart Invoice Extractor project successfully demonstrates the use of multimodal vision AI models to automate invoice data extraction. The system efficiently converts invoice images into structured JSON and tabular data while handling real-world challenges such as rate limits, invalid responses, and data verification.

This confirms that all tasks assigned for the project were successfully implemented and tested.

Discussion Questions for Tomorrow's Session

1. Right now our Smart Invoice Extractor just calls the Qwen vision model through the OpenRouter HTTP API. If in the future we want to run this vision model ourselves (self-host it) instead of using OpenRouter, what would a proper 'production-ready' deployment look like in terms of servers, GPUs, and inference setup?

2. In our invoice app, the user uploads an image and waits for the JSON and table. How should we think about 'speed' for this kind of vision model – like what exactly should we measure, and what can we do in the frontend and backend so it still feels fast to the user even if the model is a bit slow?
3. If our Smart Invoice Extractor grows into a SaaS product with thousands of invoices per day, how should we estimate and optimize cost at the inference layer (GPU choice, quantization, batching, MIG, etc.) for a multimodal model like this, based on the hardware and techniques chapters?